DSA STUDY BLUEPRINT SERIES

54. Merge Two Sorted Lists

Merge Two Sorted Lists

Section 07 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Merge:** Combine nodes iteratively in sorted order.
- Use a dummy pointer node to build the merged list.

Memory & Execution Blueprint

L1: 1 → 3 → 5 | L2: 2 → 4 ⇒ Merged: 1 → 2 → 3 → 4 → 5

C++ Implementation Reference

Standard code layout and syntax

```
Node* mergeTwoLists(Node* l1, Node* l2) {
    Node dummy(0), *tail = &dummy;
    while (l1 && l2) {
        if (l1->val < l2->val) { tail->next = l1; l1 = l1->next; }
        else { tail->next = l2; l2 = l2->next; }
        tail = tail->next;
    }
    tail->next = l1 ? l1 : l2;
    return dummy.next;
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	O(N + M)
Space Complexity	O(1)

Key Practices & Gotchas

- **Important:** Can also be implemented recursively, using O(N + M) recursion stack space.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace the re-wirings of three node pointers reversing segment `10 -> 20`:

Step	Pointers [prev, curr, next]	Pointer reassignment	Resulting node state
1	[nullptr, 10, 20]	`curr->next = prev`	10 points to nullptr
2	[10, 20, nullptr]	`curr->next = prev`	20 points to 10 (Reversed segment completed!)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Pointer Re-linking:** Modifying node transitions requires dummy nodes to isolate head pointer alterations.
- **Fast & Slow Pointers:** Useful for loop check detections, middle-node finding, and cyclic lists splits.
- **Related LeetCode Problems:** #206 Reverse Linked List, #141 Linked List Cycle, #23 Merge k Sorted Lists.